

Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт информатики и вычислительной техники
09.03.01 "Информатика и вычислительная техника"
профиль "Программное обеспечение средств
вычислительной техники и автоматизированных систем"

Кафедра прикладной математики и кибернетики

Расчетно-графическая работа по дисциплине
Алгоритмы и вычислительные методы оптимизации
Вариант 2

Выполнил:

студент гр.ИП-312

Прозоренко К.В

«5» мая 2026 г.

Новосибирск 2026 г.

1. Задание варианта

Дисциплина: Алгоритмы и вычислительные методы оптимизации

Группа: ИП312

Вариант 2: Метод искусственного базиса

Написать программу, решающую задачу линейного программирования (ЗЛП), заданную в канонической форме, методом искусственного базиса.

Программа должна:

- работать с классом простых дробей (рациональная арифметика);
- находить решение ЗЛП методом искусственного базиса;
- принимать на вход матрицы, удовлетворяющие требованиям метода;
- обрабатывать случай отсутствия решений (несовместная система);
- при бесконечном множестве решений — находить конечное число решений (больше одного).

Каноническая форма задачи линейного программирования:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

...

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

$$x_1, x_2, \dots, x_n \geq 0$$

$$Z = c_1x_1 + c_2x_2 + \dots + c_nx_n \rightarrow \max$$

Входные данные считываются из файла в виде матрицы размера $(m+1) \times (n+1)$:

- строки 1..m — коэффициенты ограничений + правая часть (b);
- строка m+1 — коэффициенты целевой функции.

2. Описание метода искусственного базиса

Метод искусственного базиса (двухфазный симплекс-метод) применяется для решения задач линейного программирования в канонической форме, когда начальный допустимый базис неизвестен.

Фаза 1. Поиск допустимого базиса

К каждому ограничению вводится искусственная переменная $a_i \geq 0$. Строится вспомогательная задача минимизации суммы искусственных переменных:

$$W = a_1 + a_2 + \dots + a_m \rightarrow \min$$

Начальный базис — искусственные переменные: $a_i = b_i \geq 0$ (при необходимости умножают строку на -1 для обеспечения неотрицательности).

Симплекс-итерации проводятся до тех пор, пока все оценки $\Delta_j \geq 0$ для небазисных переменных $x_1 \dots x_n$.

Формула оценки в фазе 1:

$$\Delta_j = -\sum \{ a_i \in \text{базис} \} a_{ij}, \quad j = 1 \dots n$$

Если в конце фазы 1 хотя бы одна искусственная переменная остаётся в базисе с ненулевым значением — система несовместна, решения нет.

Фаза 2. Оптимизация

После нахождения допустимого базиса применяется стандартный симплекс-метод для максимизации исходной целевой функции Z .

Оценки для фазы 2:

$$\Delta_j = \sum_i c_{\beta i} \cdot a_{ij} - c_j, \quad j = 1 \dots n$$

где $c_{\beta i}$ — коэффициент целевой функции при базисной переменной строки i .

Правило выбора разрешающего элемента

Входящая переменная: x_j с наименьшим $\Delta_j < 0$.

Выходящая переменная: строка с минимальным отношением:

$$\theta = \min \{ b_i / a_{ij} \mid a_{ij} > 0 \}$$

Критерии останова

Оптимальное решение: все $\Delta_j \geq 0$.

Нет решения (несовместность): после фазы 1 искусственная переменная ненулевая.

Неограниченная задача: для входящей переменной нет строк с $a_{ij} > 0$.

Бесконечно много решений: существует небазисная переменная с $\Delta_j = 0$.

Для поиска второго решения при бесконечном множестве решений программа выполняет дополнительный шаг: вводит в базис небазисную переменную с $\Delta_j = 0$.

3. Результаты работы программы

Тест 1. Единственное оптимальное решение

Входные данные:

$$x_1 + x_2 = 4$$

$$x_1 + 2x_2 = 6$$

$$Z = 2x_1 + 3x_2 \rightarrow \max$$

Вывод программы:

ФАЗА 1: Поиск базиса

Начальная таблица:

Базис	x1	x2	a1	a2	b
a1	1	1	1	0	4
a2	1	2	0	1	6
Δ	-2	-3	-	-	

Шаг 1: вводим x1, выводим a1 (элемент = 1)

Базис	x1	x2	a1	a2	b
x1	1	1	1	0	4
a2	0	1	-1	1	2
Δ	0	-1	-	-	

Шаг 2: вводим x2, выводим a2 (элемент = 1)

Базис	x1	x2	a1	a2	b
x1	1	0	2	-1	2

x2		0		1		-1		1		2
-----	+	-----	+	-----	+	-----	+	-----	+	-----
Δ		0		0		-		-		
-----	+	-----	+	-----	+	-----	+	-----	+	-----

→ Допустимый базис найден.

ФАЗА 2: Оптимизация

Таблица после фазы 1:

-----	+	-----	+	-----	+	-----	+	-----	+	-----
Базис		x1		x2		a1		a2		b
-----	+	-----	+	-----	+	-----	+	-----	+	-----
x1		1		0		2		-1		2
x2		0		1		-1		1		2
-----	+	-----	+	-----	+	-----	+	-----	+	-----
Δ		0		0		-		-		
-----	+	-----	+	-----	+	-----	+	-----	+	-----

→ Все $\Delta \geq 0$ — оптимум достигнут.

РЕЗУЛЬТАТ: Оптимальное решение:

$$x_1 = 2$$

$$x_2 = 2$$

$$Z = 10$$

Результат: $x_1 = 2, x_2 = 2, Z = 10$.

Тест 2. Несовместная система (решения нет)

Входные данные:

$$x_1 + x_2 = 1$$

$$x_1 + x_2 = 2$$

$$Z = x_1 + x_2 \rightarrow \max$$

Вывод программы:

ФАЗА 1: Поиск базиса

Начальная таблица:

Базис	x1	x2	a1	a2	b
a1	1	1	1	0	1
a2	1	1	0	1	2
Δ	-2	-2	-	-	

Шаг 1: вводим x1, выводим a1 (элемент = 1)

Базис	x1	x2	a1	a2	b
x1	1	1	1	0	1
a2	0	0	-1	1	1
Δ	0	0	-	-	

→ Искусственные переменные ненулевые — система несовместна.

РЕЗУЛЬТАТ: Решение отсутствует (система несовместна).

Результат: программа корректно определила, что система ограничений противоречива (ограничения $x_1+x_2=1$ и $x_1+x_2=2$ несовместны).

Тест 3. Бесконечное множество решений

Входные данные:

$$x_1 + x_2 = 4$$

$$Z = x_1 + x_2 \rightarrow \max$$

Вывод программы:

ФАЗА 1: Поиск базиса

Начальная таблица:

Базис	x1	x2	a1	b	
a1	1	1	1	4	
Δ	-1	-1	-		

Шаг 1: вводим x1, выводим a1 (элемент = 1)

Базис	x1	x2	a1	b	
x1	1	1	1	4	
Δ	0	0	-		

→ Допустимый базис найден.

ФАЗА 2: Оптимизация

→ Все $\Delta \geq 0$ — оптимум достигнут.

→ x2 небазисная с $\Delta=0$ — ищем второе решение (вводим x2).

РЕЗУЛЬТАТ: Бесконечно много решений. Два из них:

Решение 1:

$$x_1 = 4, \quad x_2 = 0$$

Решение 2:

$$x_1 = 0, \quad x_2 = 4$$

$$Z = 4$$

Результат: обнаружено бесконечное множество решений. Найдены два: $(x_1=4, x_2=0)$ и $(x_1=0, x_2=4)$. $Z = 4$.

Тест 4. Дробные коэффициенты

Входные данные:

$$2x_1 + x_2 = 5$$

$$x_1 + 3x_2 = 6$$

$$Z = (1/2)x_1 + (3/4)x_2 \rightarrow \max$$

Вывод программы:

ФАЗА 1: Поиск базиса

Начальная таблица:

Базис	x1	x2	a1	a2	b	
a1	2	1	1	0	5	
a2	1	3	0	1	6	
Δ	-3	-4	-	-		

Шаг 1: вводим x1, выводим a1 (элемент = 2)

Базис	x1	x2	a1	a2	b	
x1	1	1/2	1/2	0	5/2	
a2	0	5/2	-1/2	1	7/2	
Δ	0	-5/2	-	-		

Шаг 2: вводим x2, выводим a2 (элемент = 5/2)

Базис	x1	x2	a1	a2	b	
x1	1	0	3/5	-1/5	9/5	
x2	0	1	-1/5	2/5	7/5	

```

-----+-----+-----+-----+-----+-----
      Δ |  0 |  0 |      - |      - |
-----+-----+-----+-----+-----+-----

```

→ Допустимый базис найден.

ФАЗА 2: Оптимизация

→ Все $\Delta \geq 0$ — оптимум достигнут.

РЕЗУЛЬТАТ: Оптимальное решение:

$$x_1 = 9/5$$

$$x_2 = 7/5$$

$$Z = 39/20$$

Результат: $x_1 = 9/5$, $x_2 = 7/5$, $Z = 39/20$. Программа работает с рациональной арифметикой — все промежуточные и итоговые значения хранятся в виде точных дробей.

4. Код программы

simplex.go

```
package main

import (
    "ACMO/common"
    "fmt"
    "strings"
    "unicode/utf8"
)

var zero = common.NewDrob(0, 1)
var one = common.NewDrob(1, 1)

func neg(d common.Drob) common.Drob {
    return common.NewDrob(-d.Chisl, d.Znam)
}

type SimplexTable struct {
    rows int
    cols int
    tab   [][]common.Drob
    basis []int
}

func newTable(rows, cols int) *SimplexTable {
    tab := make([][]common.Drob, rows)
    for i := range tab {
        tab[i] = make([]common.Drob, cols+1)
        for j := range tab[i] {
            tab[i][j] = zero
        }
    }
    return &SimplexTable{rows: rows, cols: cols, tab: tab, basis: make([]int, rows)}
}

func (t *SimplexTable) pivot(pivRow, pivCol int) {
    pivElem := t.tab[pivRow][pivCol]
    for j := 0; j <= t.cols; j++ {
        t.tab[pivRow][j] = t.tab[pivRow][j].Divide(pivElem)
    }
}
```

```

    for i := 0; i < t.rows; i++ {
        if i == pivRow {
            continue
        }
        factor := t.tab[i][pivCol]
        if factor.Chisl == 0 {
            continue
        }
        for j := 0; j <= t.cols; j++ {
            t.tab[i][j] = t.tab[i][j].Minus(factor.Multiply(t.tab[pivRow][j]))
        }
    }
    t.basis[pivRow] = pivCol
}

```

```

func chooseLeaving(t *SimplexTable, col int) int {
    minRow := -1
    var minRatio common.Drob
    for i := 0; i < t.rows; i++ {
        aij := t.tab[i][col]
        if aij.Chisl <= 0 {
            continue
        }
        ratio := t.tab[i][t.cols].Divide(aij)
        if minRow == -1 || ratio.Compare(minRatio) < 0 {
            minRow = i
            minRatio = ratio
        }
    }
    return minRow
}

```

```

func varName(j, n int) string {
    if j < n {
        return fmt.Sprintf("x%d", j+1)
    }
    return fmt.Sprintf("a%d", j-n+1)
}

```

```

func printTableau(t *SimplexTable, n int, delta []common.Drob, pivRow, pivCol int) {
    total := t.cols

```

```
rln := utf8.RuneCountInString
colCount := total + 2
widths := make([]int, colCount)
```

```
header := make([]string, colCount)
header[0] = "Базис"
for j := 0; j < total; j++ {
    header[j+1] = varName(j, n)
}
header[colCount-1] = "b"
for i, h := range header {
    if rln(h) > widths[i] {
        widths[i] = rln(h)
    }
}
```

```
cells := make([][]string, t.rows)
for i := 0; i < t.rows; i++ {
    cells[i] = make([]string, colCount)
    cells[i][0] = varName(t.basis[i], n)
    for j := 0; j < total; j++ {
        cells[i][j+1] = t.tab[i][j].ToString()
    }
    cells[i][colCount-1] = t.tab[i][total].ToString()
    for k, s := range cells[i] {
        if rln(s) > widths[k] {
            widths[k] = rln(s)
        }
    }
}
```

```
deltaRow := make([]string, colCount)
deltaRow[0] = "Δ"
if delta != nil {
    for j := 0; j < n; j++ {
        deltaRow[j+1] = delta[j].ToString()
    }
    for j := n; j < total; j++ {
        deltaRow[j+1] = "-"
    }
}
deltaRow[colCount-1] = ""
```

```

for k, s := range deltaRow {
    if rlen(s) > widths[k] {
        widths[k] = rlen(s)
    }
}

```

```

pad := func(s string, w int) string {
    return strings.Repeat(" ", w-rlen(s)) + s
}
hline := func() {
    parts := make([]string, colCount)
    for i, w := range widths {
        parts[i] = strings.Repeat("-", w+2)
    }
    fmt.Println(strings.Join(parts, "+"))
}
printRow := func(row []string, markCol int) {
    parts := make([]string, colCount)
    for i, s := range row {
        cell := " " + pad(s, widths[i]) + " "
        if markCol >= 0 && i == markCol+1 {
            cell = "[" + pad(s, widths[i]) + "]"
        }
        parts[i] = cell
    }
    fmt.Println(strings.Join(parts, "|"))
}

```

```

hline()
printRow(header, -1)
hline()
for i, row := range cells {
    mc := -1
    if i == pivRow && pivCol >= 0 {
        mc = pivCol
    }
    printRow(row, mc)
}
hline()
printRow(deltaRow, -1)
hline()
}

```

```
const (
    Optimal      = 0
    Infeasible   = 1
    Unbounded    = 2
    MultiSol     = 3
)
```

```
type LPResult struct {
    Code      int
    Solutions [][]common.Drob
    ObjVal     common.Drob
}
```

```
func SolveLP(A [][]common.Drob, b []common.Drob, c []common.Drob) LPResult {
    m := len(A)
    n := len(c)
    total := n + m
```

```
    t := newTable(m, total)
```

```
    for i := 0; i < m; i++ {
        sign := one
        if b[i].Chisl < 0 {
            sign = neg(one)
        }
        for j := 0; j < n; j++ {
            t.tab[i][j] = A[i][j].Multiply(sign)
        }
        t.tab[i][n+i] = one
        t.tab[i][total] = b[i].Multiply(sign)
        t.basis[i] = n + i
    }
```

```
    phase1Delta := func() []common.Drob {
        delta := make([]common.Drob, n)
        for j := 0; j < n; j++ {
            s := zero
            for i := 0; i < m; i++ {
                if t.basis[i] >= n {
                    s = s.Sum(t.tab[i][j])
                }
            }
        }
    }
```

```

    }
    delta[j] = neg(s)
  }
  return delta
}

```

```

fmt.Println(" ")
fmt.Println("          ФАЗА 1: Поиск базиса ")
fmt.Println(" ")
fmt.Println("\nНачальная таблица:")
printTableau(t, n, phase1Delta(), -1, -1)

```

```

step := 1
for {
    delta := phase1Delta()
    entering := -1
    for j := 0; j < n; j++ {
        if delta[j].Chisl < 0 {
            entering = j
            break
        }
    }
    if entering == -1 {
        break
    }
    leaving := chooseLeaving(t, entering)
    if leaving == -1 {
        break
    }
    fmt.Printf("\nШаг %d: вводим %s, выводим %s (элемент = %s)\n",
        step, varName(entering, n), varName(t.basis[leaving], n),
        t.tab[leaving][entering].ToString())
    t.pivot(leaving, entering)
    step++
    printTableau(t, n, phase1Delta(), -1, -1)
}

```

```

for i := 0; i < m; i++ {
    if t.basis[i] >= n && t.tab[i][total].Chisl != 0 {
        fmt.Println("\n→ Искусственные переменные ненулевые — система несовместна.")
        return LPResult{Code: Infeasible}
    }
}

```



```

}
fmt.Println("\n→ Допустимый базис найден.")

```

```

phase2Delta := func() []common.Drob {
    delta := make([]common.Drob, n)
    for j := 0; j < n; j++ {
        s := zero
        for i := 0; i < m; i++ {
            bv := t.basis[i]
            var cb common.Drob
            if bv < n {
                cb = c[bv]
            } else {
                cb = zero
            }
            s = s.Sum(cb.Multiply(t.tab[i][j]))
        }
        delta[j] = s.Minus(c[j])
    }
    return delta
}

```

```

fmt.Println("\n" + " ")
fmt.Println(" " + "    ФАЗА 2: Оптимизация    " + " ")
fmt.Println(" " + " " + " ")
fmt.Println("\nТаблица после фазы 1:")
printTableau(t, n, phase2Delta(), -1, -1)

```

```

step = 1
for {
    delta := phase2Delta()
    entering := -1
    for j := 0; j < n; j++ {
        if delta[j].Chisl < 0 {
            entering = j
            break
        }
    }
    if entering == -1 {
        break
    }
    leaving := chooseLeaving(t, entering)
}

```

```

    if leaving == -1 {
        fmt.Println("\n→ Целевая функция не ограничена.")
        return LPResult{Code: Unbounded}
    }
    fmt.Printf("\nШаг %d: вводим %s, выводим %s (элемент = %s)\n",
        step, varName(entering, n), varName(t.basis[leaving], n),
        t.tab[leaving][entering].ToString())
    t.pivot(leaving, entering)
    step++
    printTableau(t, n, phase2Delta(), -1, -1)
}
fmt.Println("\n→ Все  $\Delta \geq 0$  – оптимум достигнут.")

```

```

extractSol := func() []common.Drob {
    x := make([]common.Drob, n)
    for j := range x {
        x[j] = zero
    }
    for i := 0; i < m; i++ {
        if t.basis[i] < n {
            x[t.basis[i]] = t.tab[i][total]
        }
    }
    return x
}

```

```

sol1 := extractSol()
objVal := zero
for j := 0; j < n; j++ {
    objVal = objVal.Sum(c[j].Multiply(sol1[j]))
}

```

```

delta := phase2Delta()
altSols := [][]common.Drob{sol1}

```

```

for j := 0; j < n; j++ {
    if delta[j].Chisl != 0 {
        continue
    }
    isBasic := false
    for i := 0; i < m; i++ {
        if t.basis[i] == j {

```

```

        isBasic = true
        break
    }
}
if isBasic {
    continue
}
leaving := chooseLeaving(t, j)
if leaving == -1 {
    continue
}
fmt.Printf("\n→ %s небазисная с  $\Delta=0$  – ищем второе решение (вводим %s).\n",
    varName(j, n), varName(j, n))
t.pivot(leaving, j)
sol2 := extractSol()
obj2 := zero
for k := 0; k < n; k++ {
    obj2 = obj2.Sum(c[k].Multiply(sol2[k]))
}
if obj2.Compare(objVal) == 0 {
    altSols = append(altSols, sol2)
}
break
}
}

```

```

code := Optimal
if len(altSols) > 1 {
    code = MultiSol
}
return LPResult{Code: code, Solutions: altSols, ObjVal: objVal}
}

```

```

func printResult(r LPResult, n int) {
    fmt.Println("\n=====")
    switch r.Code {
    case Infeasible:
        fmt.Println("РЕЗУЛЬТАТ: Решение отсутствует (система несовместна).")
    case Unbounded:
        fmt.Println("РЕЗУЛЬТАТ: Целевая функция не ограничена.")
    case Optimal:
        fmt.Println("РЕЗУЛЬТАТ: Оптимальное решение:")
        printSol(r.Solutions[0], n)
    }
}

```

```

    fmt.Printf("Z = %s\n", r.ObjVal.ToString())
case MultiSol:
    fmt.Println("РЕЗУЛЬТАТ: Бесконечно много решений. Два из них:")
    fmt.Println("\nРешение 1:")
    printSol(r.Solutions[0], n)
    fmt.Println("\nРешение 2:")
    printSol(r.Solutions[1], n)
    fmt.Printf("\nZ = %s\n", r.ObjVal.ToString())
}
fmt.Println("=====")
}

func printSol(x []common.Drob, n int) {
    for j := 0; j < n; j++ {
        fmt.Printf("  x%d = %s\n", j+1, x[j].ToString())
    }
}

```

main.go

```

package main

import (
    "ACMO/common"
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "Использование: rgz <файл_входных_данных>")
        os.Exit(1)
    }

    A, b, c, err := readInput(os.Args[1])
    if err != nil {
        fmt.Fprintf(os.Stderr, "Ошибка чтения файла: %v\n", err)
        os.Exit(1)
    }
}

```

```

    n := len(c)
    result := SolveLP(A, b, c)
    printResult(result, n)
}

```

```

func readInput(path string) (A [][]common.Drob, b []common.Drob, c []common.Drob,
err error) {
    f, err := os.Open(path)
    if err != nil {
        return
    }
    defer f.Close()

```

```

    var rows [][]common.Drob
    scanner := bufio.NewScanner(f)
    for scanner.Scan() {
        line := strings.TrimSpace(scanner.Text())
        if line == "" || strings.HasPrefix(line, "#") {
            continue
        }
        parts := strings.Fields(line)
        row := make([]common.Drob, len(parts))
        for i, p := range parts {
            row[i], err = parseDrob(p)
            if err != nil {
                return
            }
        }
        rows = append(rows, row)
    }
    if scanner.Err() != nil {
        err = scanner.Err()
        return
    }
    if len(rows) < 2 {
        err = fmt.Errorf("недостаточно строк в файле")
        return
    }
}

```

```

    m := len(rows) - 1
    n := len(rows[0]) - 1

```

```

A = make([][]common.Drob, m)
b = make([]common.Drob, m)
for i := 0; i < m; i++ {
    if len(rows[i]) != n+1 {
        err = fmt.Errorf("несовместимая длина строки %d", i+1)
        return
    }
    A[i] = rows[i][:n]
    b[i] = rows[i][n]
}

if len(rows[m]) < n {
    err = fmt.Errorf("строка целевой функции слишком короткая")
    return
}
c = rows[m][:n]
return
}

func parseDrob(s string) (common.Drob, error) {
    parts := strings.SplitN(s, "/", 2)
    chisl, err := strconv.Atoi(parts[0])
    if err != nil {
        return common.Drob{}, fmt.Errorf("неверное число: %s", s)
    }
    if len(parts) == 1 {
        return common.NewDrob(chisl, 1), nil
    }
    znam, err := strconv.Atoi(parts[1])
    if err != nil {
        return common.Drob{}, fmt.Errorf("неверное число: %s", s)
    }
    return common.NewDrob(chisl, znam), nil
}

```

drob.go

```

package common

import (
    "fmt"
)

```

```
type Drob struct {  
    Chisl int  
    Znam  int  
}
```

```
func NewDrob(Chisl, Znam int) Drob {  
    if Znam == 0 {  
        panic("знаменатель не должен быть 0")  
    }  
    if Znam < 0 {  
        Znam = -Znam  
        Chisl = -Chisl  
    }  
    divisor := gcd(Chisl, Znam)
```

```
    return Drob{Chisl: Chisl / divisor, Znam: Znam / divisor}  
}
```

```
func gcd(a, b int) int {  
    a = abs(a)  
    b = abs(b)  
    for b != 0 {  
        a, b = b, a%b  
    }  
    return a  
}
```

```
func abs(x int) int {  
    if x < 0 {  
        return -x  
    }  
    return x  
}
```

```
func (d Drob) Multiply(other Drob) Drob {  
    return NewDrob(  
        d.Chisl*other.Chisl,  
        d.Znam*other.Znam)
```

```
    }  
    func (d Drob) Divide(other Drob) Drob {  
        return NewDrob(  
            d.Chisl*other.Znam,  
            d.Znam*other.Chisl)
```

```

}
func (d Drob) Sum(other Drob) Drob {
    newChisl := d.Chisl*other.Znam + other.Chisl*d.Znam
    newZnam := d.Znam * other.Znam
    return NewDrob(newChisl, newZnam)
}
func (d Drob) Minus(other Drob) Drob {
    newChisl := d.Chisl*other.Znam - other.Chisl*d.Znam
    newZnam := d.Znam * other.Znam
    return NewDrob(newChisl, newZnam)
}

```

```

func (d Drob) Abs() Drob {
    if d.Chisl < 0 {
        d.Chisl = -d.Chisl
    }
    return d
}

```

```

}
func (d Drob) Compare(other Drob) int {
    diff := d.Minus(other)
    if diff.Chisl > 0 {
        return 1
    } else if diff.Chisl < 0 {
        return -1
    }
    return 0
}
func (d Drob) ToString() string {
    if d.Znam == 1 || d.Chisl == 0 {
        return fmt.Sprintf("%d", d.Chisl)
    }
    return fmt.Sprintf("%d/%d", d.Chisl, d.Znam)
}

```